Sistemas de Informação



Disciplina: Programação I

Prof. Renê Rodrigues Veloso

Generics

... na aula passada implementamos uma TAD Lista Encadeada que armazena valores inteiros ...

Mas... a pergunta óbvia aqui é:

Se quisermos armazenar valores de outros tipos, precisaremos criar uma classe e uma interface para cada novo tipo?

Um problema simples

```
public class ValorInteiro {
   private Integer i;

   public ValorInteiro(Integer i) {
      this.i = i;
   }

   public void imprime() {
      System.out.println(“Valor = “+i);
   }
}
```

Um problema simples

```
public class ValorReal {
   private Double i;

   public ValorReal(Double i) {
      this.i = i;
   }

   public void imprime() {
      System.out.println(“Valor = “+i);
   }
}
```

Um problema simples

```
public class ValorString {
   private String i;

   public ValorString(String i) {
      this.i = i;
   }

   public void imprime() {
      System.out.println(“Valor = “+i);
   }
}
```

Uma solução simples

```
public class Valor {
   private Object i;

   public Valor(Object i) {
      this.i = i;
   }

   public void imprime() {
      System.out.println("Valor = "+i);
   }
}
```

Uma solução simples

```
class Main {
  public static void main(String[] args) {

    Valor it = new Valor(4);
    Valor dt = new Valor(1.5);


    dt.imprime();
    it.imprime();
  }
}
```

Generics

Permite também a definição de **classes genéricas**

- Os atributos da classe podem ser definidos no momento da instanciação do objeto
- Recurso útil ao definir classes como estruturas de dados

Permite aos programadores escreverem **métodos genéricos**

- Os parâmetros dos métodos, variáveis locais e o tipo de retorno podem ser definidos na chamada do método
- Permite ao mesmo método ser invocado usando-se tipos distintos (sem precisar sobrescrevê-lo)

Oferece os mesmos recursos dos Templates em C++

Uma solução simples

```
public class Valor<T> {
   private T i;

   public Valor(T i) {
      this.i = i;
   }

   public void imprime() {
      System.out.println(“Valor = “+i);
   }
}
```

Uma solução simples

```
class Main {
  public static void main(String[] args) {
    Valor<Integer> it = new Valor<Integer>(4);
    Valor<Double> dt = new Valor<Double>(1.5);
    Valor<String> st = new Valor<String>("oi");

    dt.imprime();
    it.imprime();
    st.imprime();
  }
}
```

Convenções de nomes

Por definição, qualquer nome válido para identificador pode ser usado, mas é comum o uso de letras maiúsculas para os tipos genéricos:

E - Element (muito usado no framework Java Collections)
K - Key (chave, no caso de pares chave-valor)
N - Number
T - Type
V - Value
S,U,V etc. - 2nd, 3rd, 4th tipos

Múltiplos tipos

```
class name<T1, T2, ..., Tn> { /* ... */ }


public class Box<T1, T2> {
    private T1 t1;
    private T2 t2;

     ...
    public void setT1(T1 t) { this.t1 = t; }
    public T2 get() { return t2; }
}
```

Múltiplos tipos

```
public interface Pair<K, V> {
    public K getKey();
    public V getValue();
}
```

Múltiplos tipos

```
public class OrderedPair<K, V> implements Pair<K, V>
{
    private K key;
    private V value;

    public OrderedPair(K key, V value) {
      this.key = key;
      this.value = value;
    }

    public K getKey() { return key; }
    public V getValue() { return value; }
}
```

Múltiplos tipos

```
class Main {
 public static void main(String[] args) {
  Pair<String,Integer> p1 =
            new OrderedPair<String,Integer>("par",8);

  Pair<String,String>  p2 =
          new OrderedPair<String,String>("oi","mundo");

  Pair<String,Box<Integer,Double>> p3 =
     new OrderedPair<String,String>(1, new Box<Integer,Double>(...));
 }
}
```

Atenção:

Métodos genéricos (e classes genéricas) podem ser definidos apenas para tipos referenciáveis

Logo, não podem ser definidos para tipos primitivos

{byte, short, int, long, float, double, boolean, char}

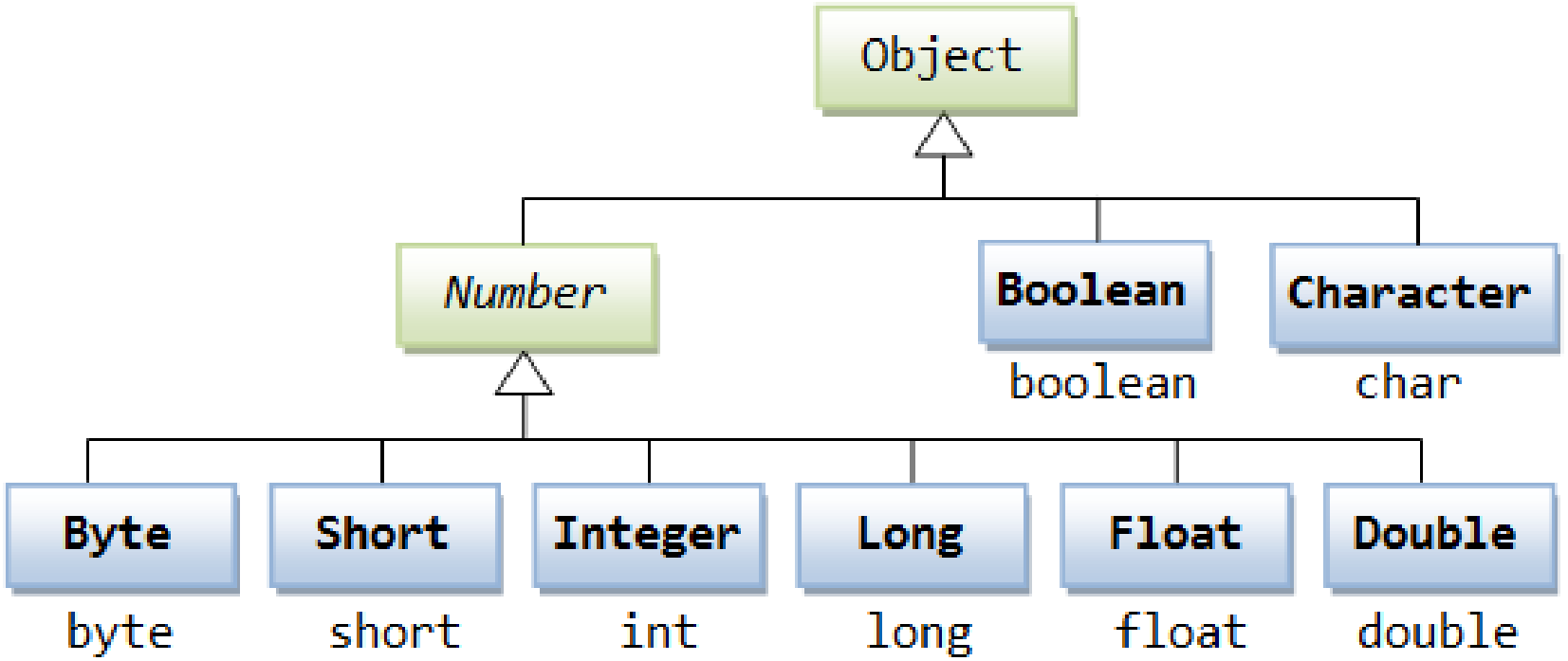
Essa limitação é contornada usando-se as **classes empacotadoras (wrappers) de tipo**, que são uma alternativa oferecida por Java para tratar tipos primitivos como referenciáveis de forma transparente

{Byte, Short, Integer, Long, Float, Double, Boolean, Character}

Forçando a checagem de tipo

Problema:

Como fazemos para garantir que a nossa classe Valor aceite apenas valores numéricos?



Uma solução simples

```
public class Valor<T extends Number> {
   private T i;

   public Valor(T i) {
      this.i = i;
   }

   public void imprime() {
      System.out.println("Valor = "+i);
   }
}
```

Interfaces em Java

```
public interface List<T>{
  void append(T valor);
  void insert(T valor);
  void insert(T valor, int pos);
  T get(int pos);
  int size();
}
```

Interfaces em Java

```
public class ListaEncadeadaSimples<T> implements List<T>{
  private No<T> inicio;
  private No<T> fim;
  private int tamanho;

  public ListaEncadeadaSimples(){
    tamanho = 0;
    inicio = fim = null;
  } . . .
```

Interfaces em Java

```
public class No<T> {
  private T valor;
  private No<T> prox;

  public No() {
    this(0);
  }

  public No(T valor){
    this.valor = valor;
    prox = null;
  } . . .
```